

Xinu Pi 上の並列実行方式の比較

ワーカーズ型（郵便箱・静的分割）対 対称 SMP 型（共有 ready キュー）

— N-Queens・通常の並列計算・AIPL アクター並列の実測（Pi 5 / Pi 4 / Pi 3）

児玉研究室

2026 年 9 月 10 日

概要

Raspberry Pi 5 / 4 / 3 上のベアメタル Xinu に、並列実行の二方式 — 核 0 が各二次コアの郵便箱へ仕事を静的に配る**ワーカーズ型**と、各コアが共有 ready キューから自分で取る**対称 SMP 型**— を実装し、N-Queens・通常の並列計算・**AIPL のアクター並列**で比較した。

ワーカーズ型については測定が完結した。静的分割の makespan モデルは実測に 5% 前後で追随し、**A76 と A72 で比の曲線がほぼ一致する**。静的分割の弱点は「静的であること」ではなく**分割の粗さ**であり、重い仕事が散らばっていれば分割数を増やすだけで理想に近づく（301.5 → 235.3 ms）。仕事を完全に等量にしても 4 コアで 4 倍は出ず（2→4 コアで 1 反復が +27.7%）、N-Queens が 2.2× 止まりだった主因はここにある。

一方、対称 SMP 型と AIPL の比較は取れなかった — その経路が正しく動かないためである。検算計器を入れると誤答・停止・板の死が現れた。時間の行き先を分解した結果、原因は**呼び手が自分の持ち分を計算すると先取りで 1 ティックずつ失う**ことにあり（399 回の先取り × 10.4 ms = 4.15 秒、inline_us と一致）、生成した単位そのものは 1.85 ms で完了していた。静的分割では自然な「呼び手も 1 単位ぶん働く」設計が、動的スケジューラの上では 20 倍の劣化を招く。**初稿の「不揃いな仕事では対称 SMP 型が 1.25 倍速い」は撤回する**。

測定の過程で、例外処理が ELR_EL1/SPSR_EL1 を保存していないというカーネルの重大な不具合を Pi 5 と Pi 4 の双方で発見・修正した（先取りを有効にすると 8 プロセス中 8 個すべてが誤った答えを返す）。Pi 3 は ARM32 で、同じ状態をスレッド自身のスタックに積んでおり、この穴は無かった。また**自分の測定手順にも 3 つの穴**（正誤の未確認・標本が 2 ラウンドで停止・冷間始動の過渡）を見つけ、いずれも計器を足して塞いだうえで全実験を測り直した。

1 目的と仮説

我々の研究は、Xinu Pi の上で AIPL を実行することを目的としている。そのためのマルチプロセッサ構成として、**ワーカーズ型が効率的である**というのが作業仮説である。本報告はこの仮説を実機で検証する。

主張を「速い」で終わらせず、**なぜ速いのか**を機構で説明できる形にした。すなわち 1 メッセージあたりの費用を

$$\text{us/msg} = \underbrace{a}_{\text{配布コスト}} + \underbrace{b \times (\text{粒度})}_{\text{本体の実行}}$$

と分解し、両方式で a を比較する — という筋書きで始めた。 b （本体の実行速度）は方式によらず同じはずで、実測でも一致した。

しかしこの筋書きは途中で成り立たなくなった。検算の計器を入れたところ、**対称 SMP 型は**

AIPL を正しく走らせられないことが分かったからである（第 15 節）。 a の比較には両方が正しく動くことが要る。そこで本報告は次の 2 つに分けて書く。

- **ワーカーズ型については、定常値で測り切る。**静的分割の makespan モデルがどこまで実測を説明するか、その弱点は何か、天井はどこかを三板で確かめる。
- **対称 SMP 型については、なぜ動かないのかを機構で説明する。**「動かなかった」で終わらせず、時間の行き先を分解して原因を数で特定する。その原因自体が、短い仕事を汎用スケジューラに載せる費用であり、間接的ではあるが仮説を支持する材料になる。

なお本報告には自分の測定手順の誤りとその訂正、および外れた仮説も、経過のまま記す。実機の測定では、どの計器を信じてよいかを確かめる過程そのものが結果の一部だからである。

2 二方式の実装

2.1 ワーカーズ型（郵便箱・静的分割）

核 0 が各二次コアの郵便箱（`smp_job_*`）に「関数と区間」を置き、SEV で起こす。二次コアは `smp_worker_loop` で待っており、受け取った区間を実行して結果を書き戻す。区間はあらかじめ均等に割る（静的分割）。核 0 も自分の区間をその場で実行する。プロセス生成も文脈切替もスケジューラのロックも要らない。実装は `system/smp.c` の `smp_parallel_sum`。

2.2 対称 SMP 型（共有 ready キュー）

仕事の単位ごとに Xinu プロセスを作って共有 ready キューへ入れ、各コアが `smpsched_core_loop` で自分で取る。偏りは走行中に均される。実装は `system/smpsched.c`。AIPL のアクター・バッチ用に汎用の `smpsched_parallel(fn, n, nunits)` を新設した。

2.3 同一カーネルでの切り替え

二方式を別々のカーネルにすると、温度・過渡・環境が交絡する。そこで**起動時は必ずワーカーズ型で上がり**、`/smpmode?on=1` を叩いた時点で二次コアが共有 ready キューへ移る作りにした（一方通行。戻すには電源再投入）。これにより同じ起動のまま A/B が取れる。

なお当初は対称スケジューラが二次コアの起動先そのものだったため、そこに不具合があると**板ごと起動しなかった**。危険な機構を起動経路から外したのは、実験を回すための前提整備でもある。

3 測定の作法

1. **計器を先に確かめる。**スピンロックの自己診断（4 コア × 20000 回で期待値 80000 と一致）と、いまどちらの方式かを毎回確認してから測る。
2. **正しさを毎回検算する。**N-Queens は既知の解数（ $n = 13$ で 73,712）と、AIPL 標本は合計値と突き合わせる。**速いが間違っている**を弾くため。
3. **中央値と最小・最大で示す**（平均では分布が見えない）。
4. **暖機を捨てる。**起動直後の過渡を数に入れない。

4 測定の作法の破れ — 自分の測定に見つけた 3 つの穴

初稿の AIPL 測定には穴が 3 つあった。いずれも「N-Queens 側では最初からやっていたのに、AIPL 側だけ抜けていた」種類の落ち度である。本節で先に述べておく — 以降の数値がすべてこの修正を経ているためである。

4.1 穴 1: 正誤を一度も外から確かめていなかった

N-Queens は毎回 `solutions=73712` と `agree=yes` を突き合わせていた。一方 AIPL の標本は各ラウンドで合計を照合し、正なら色 2・誤なら色 4 の線を引くが、その色は画面にしか出ない。この板は HDMI が無いので、誰も見ていない。

手当て: 標本を変えずに済む形として、カーネル側で線の色を数える (LINE オペコードで `col==2` なら `ok++`、`col==4` なら `ng++`)。/avm-par に `ok=/ng=` を出し、`ok>0` かつ `ng==0` でなければ採用しないという規則にした。既存の標本 39 本すべてにそのまま効く。

4.2 穴 2: 標本が 2 ラウンドで止まっていた

Main.tick は読み込み時に一度だけ積まれ、WAIT は「フレームの区切り」を立てるだけで tick を積み直さない。avm_tick を呼ぶのは WM の描画ループ `vmgfx_draw` だけだが、HDMI 無しの構成では `wm_run()` が呼ばれない (第 7 節)。つまり読み込み直後の 2 ラウンドしか走っていなかった。「6 秒の窓で回して平均を取る」という初稿の記述は実態と違う。

手当て: 駆動源 /avm-run?ms=*N* を入れた。背景プロセスにはせず、測定窓を明示的に指定する口にした — 「何ミリ秒ぶん回したか」が測定値と一対一で対応する。待ち行列が空なら tick を積み直す。

この修正で、初稿の「AVM のバッチ化が 32 アクターで破綻している」(旧・第 10 節) も誤りだったと分かった。*M* = 32 が特別だったのではなく、どの標本も 2 ラウンドで止まっていただけである。

4.3 穴 3: その 2 ラウンドは冷間始動の過渡だった

駆動源を入れて回すと、1 バッチの所要時間がバッチ数とともに増えて収束する。

測り方	AiplSpread1 の 1 バッチ
載せた直後の 2 バッチ (初稿の方式)	93.1 ms
5 バッチ	148.9 ms
19 バッチ	177.1 ms
34 バッチ	183.7 ms
暖機を捨てた定常値 (窓 2000 / 4000 ms)	186.1 / 186.0 ms

定常値は窓を変えても 0.05% 以内で一致する。初稿の AIPL の絶対値は、すべて定常値の約半分だった。過渡が定常のちょうど半分になる理由は特定できていない (第 17 節)。描画リストの蓄積を

疑って `cls()` を入れた標本で試したが差は出ず、この仮説は外れた。

手当て: 載せる → 暖機して捨てる → 計器を戻す → 測る、の順に固定した。

5 発見した不具合とその影響

測定中、計算プロセスが2本以上同時に走ると計算結果そのものが壊れる現象に遭遇した。値は毎回異なり、1本なら常に正しかった。原因は `system/exception_vectors.S` の例外入口が `ELR_EL1` (戻り番地) と `SPSR_EL1` (戻り状態) を保存していないことであった。この核は割り込みハンドラの中で文脈切替を行う (`irq_dispatch_c` → `proc_preempt` → `proc_resched` → `ctxsw`) ため、「戻り先が未確定の例外」が同時に複数存在しうる。保存していないと後から入った例外が `ELR_EL1` を上書きし、タスクが別の番地へ `eret` して計算がずれる。板は落ちず、結果だけが静かに壊れる。

修正後、 $n = 13$ の解は全構成で 73,712 に一致し、同じタスク内で二度計算した値も一致した (計器 `mism=0`)。修正前に採った測定値はすべて破棄した。本報告の数値はすべて修正後 (ビルド Sep 9 2026 19:24:47) のものである。なおこの不具合はワーカーズ型にも影響しており、N-Queens 4 コアの速度向上は $1.86\times \rightarrow 2.22\times$ に改善した。

6 結果 1: N-Queens (不揃いな探索木)

$n = 13$ 、解 73,712。直列 686.6 ms。同一カーネル・同一起動。

方式	時間 [ms]	速度向上
ワーカーズ型 1 コア	686.0	1.00
ワーカーズ型 2 コア	370.0	1.86
ワーカーズ型 3 コア	316.8	2.17
ワーカーズ型 4 コア	308.9	2.22
対称 SMP 型 (8 単位, 核 0 直接あり)	546	1.26
対称 SMP 型 (2 単位, 最良)	541	1.27
対称 SMP 型 (4 単位, 従来配布)	696	0.99

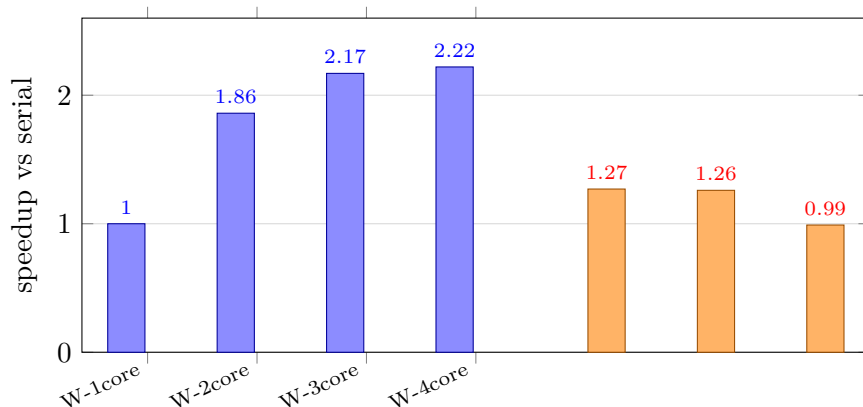


図 1: N-Queens $n = 13$ の速度向上。青=ワーカーズ型、橙=対称 SMP 型。

この表の対称 SMP 側は核 0 がスケジューラに参加していない状態の値である（計器 `ran_per_core` の先頭が常に 0 = 実質 3 コア）。これは実装の穴であり、第 7 節で直した。直したあとの 4 コア対 4 コアの比較は第 7 節を見よ（ワーカーズ型 298.8 ms 対対称 SMP 型 397 ms、差は 1.66 倍から 1.33 倍へ縮んだ）。

7 結果 1b: 核 0 をスケジューラに参加させる（実験 1）

7.1 原因は 2 段構えで、途中まで私の読みは誤っていた

「核 0 が参加していない」ことは計器で分かっていたが、どこで参加させるかを二度取り違えた。順に書く。実測でしか分からなかったからである。

1. `hook` を置いた関数が呼ばれていなかった。核 0 は描画ループ `wm_run()` に居ると考えて、そこへ引き取り口を置いた。しかしこの板は HDMI が無く、`kernel_main` は `shell_main()` へ落ちる。`wm_run()` は呼び出し元が存在しない。`hook` は一度も走っていないかった。
2. 移した先も回らなかった。核 0 の暇は `uart_getc()` の RX 空回りだと考えて移したが、シリアルアダプタが TX→RX ループバックのため RX FIFO がほとんど空にならない。計器 `poll_calls` は HTTP 要求あたり約 1 回しか増えず、`poll_hits` は 0 のままだった。
3. 正解は `proc_preempt()`。ここは 100 Hz のタイマ IRQ から EOI 後に必ず呼ばれる。しかも既存の `if (!IS_IDLE(currpid)) proc_resched();` という判定こそが「核 0 は idle だから何もしない」＝不参加の原因そのものだった。

なお `core_hits` (AIPL メッセージをコア別に何通実行したか) を足して分かったのだが、**AIPL** の配布経路では核 0 は最初から働いていた。`smpsched_parallel` の「呼び手が最後の 1 単位を直接走らせる」ぶんが核 0 で実行されるからである。核 0 不参加は `/smpsched` (N-Queens 駆動部) に固有の穴であって、AIPL の配布経路の穴ではない。初稿の記述をここで訂正する。

7.2 直した結果

	修正前 (build 21:09:08)	修正後 (build 21:37:48)
<code>ran_per_core</code> (4 単位)	0/1/2/1	1/1/1/1
N-Queens $n = 13$ 最良	496 ms	397 ms
核 0 が取った本数 <code>poll_hits</code>	0	13

4 コア対 4 コア	時間 [ms]	直列 660.1 ms からの速度向上
ワーカーズ型 4 コア	298.8	2.21
対称 SMP 型 (4 単位, 最良)	397	1.66

N-Queens ではワーカーズ型が 1.33 倍速い。修正前の 1.66 倍から縮んだので、差の約半分は核 0 の不参加という実装の穴が作っていたことになる。残る 1.33 倍が方式そのものの差である。

8 結果 2: 通常の並列計算（負荷の性質による差）

ワーカーズ型・4 コア・同一起動。いずれも直列と並列で結果が一致 (agree=yes)。

負荷	直列 [ms]	4 コア [ms]	速度向上
dining (独立卓・均一)	95.59	23.92	4.00
primes 30 万 (ほぼ均一)	41.75	13.83	3.02
nqueens $n=13$ (木が不揃い)	571.08	286.53	1.99
fill 20 万語 (メモリ律速・短い)	1.04	1.38	0.75

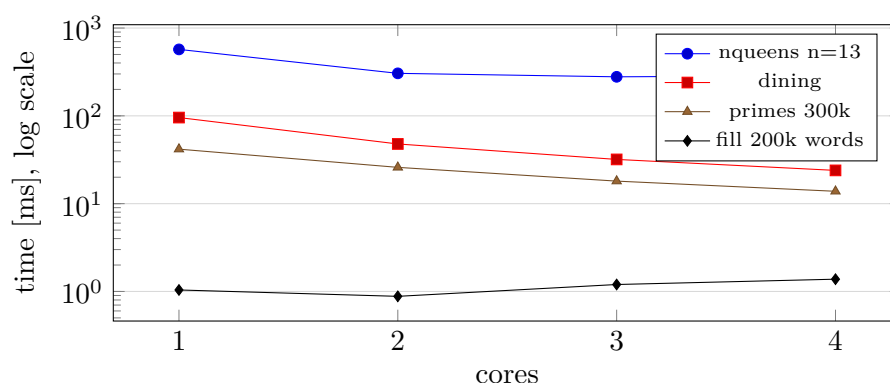


図 2: ワーカーズ型のコア数掃引。均一な仕事は 4 コアぶん出る一方、木が不揃いな nqueens は 3 コアで頭打ちになり、メモリ律速の fill は伸びない。

静的分割は、仕事が均一なら理想に近い (dining で 4.00 $\times$)。逆に不揃いだと一番重い区間を引いたコアの終了時刻が全体を決め、頭打ちになる。これは静的分割の原理的な限界であり、対称 SMP 型が本来有利になりうる領域である。

9 結果 3: AIPL アクター並列（本命）

この節の数値についての断り。本節の測定は駆動源を入れる前に採ったもので、第 4 節で述べたとおり読み込み直後の 2 ラウンド（冷間始動の過渡）しか捉えていない。定常値は約 2 倍である。また正誤を確認していない。したがって絶対値は採用しない。一方、同じ条件で二方式を交互に測った比と、コア別の実行数 `core_hits` が示す構造的な事実は、定常値で測り直した第 11 節・第 13 節と整合しており、そのまま残す。

9.1 測り方

AIPL 標本 (§18) は 8 個の Worker が同じ量の計算をする。宛先が互いに異なり `par_safe` (WAIT も描画命令も含まない) なので、実行時系はこの 8 通を 1 つのバッチにまとめて配る。粒度 (1 メッセージあたりのループ回数) を 1 から 200,000 まで 9 段階に振り、両方式で並列配布に費やした総時間 `us` とその間に配ったメッセージ数 `msgs` を計器から読み、`us/msgs` を 1 メッセージあたりの費用とした。各点 2 回、再現性は $\pm 0.1\%$ 未満。

9.2 測定値

粒度 [反復/msg]	ワーカーズ型 [$\mu\text{s}/\text{msg}$]	対称 SMP 型 [$\mu\text{s}/\text{msg}$]	比
1	3.56	5.31	1.49
10	8.94	11.69	1.31
50	36.72	39.63	1.08
200	141.91	144.81	1.02
1,000	699.50	704.09	1.01
5,000	3,487.06	3,505.88	1.005
20,000	13,954.62	14,005.00	1.004
100,000	69,764.09	70,009.50	1.004
200,000	139,695.69	140,007.62	1.002

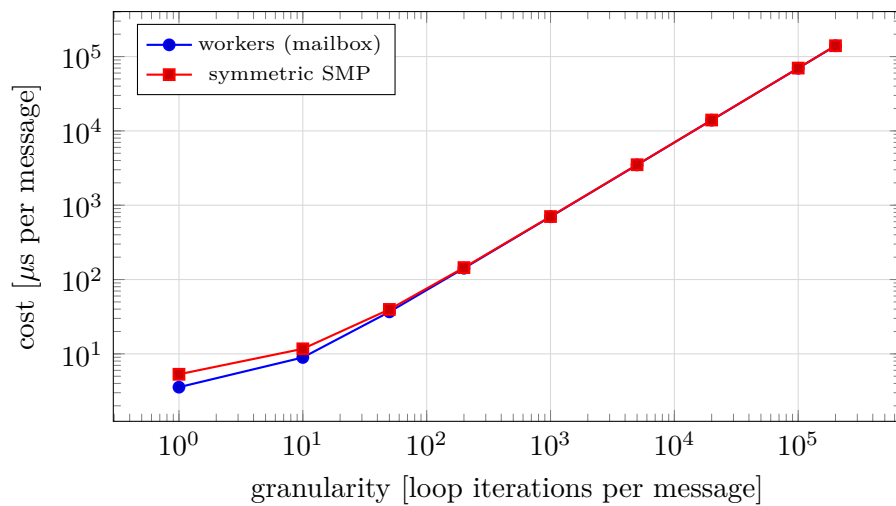


図 3: AIPL のメッセージ 1 通あたりの費用。粒度が大きい領域では両者は重なる（本体の実行が支配的）。細かい領域で分かれる。

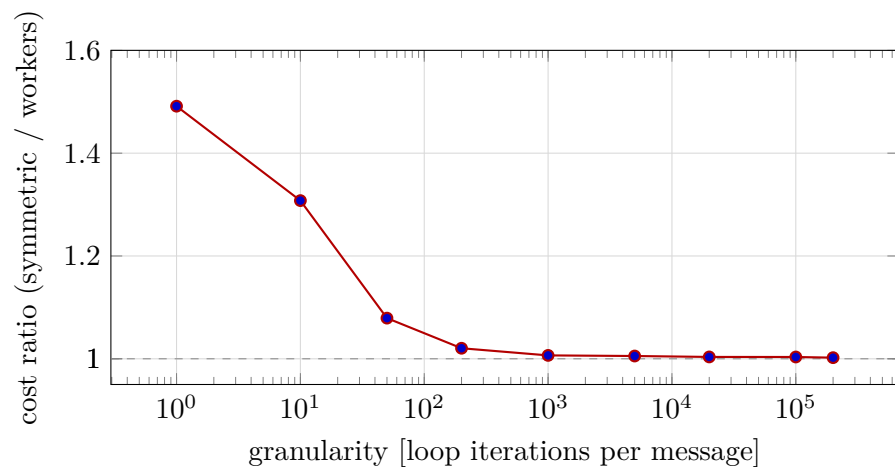


図 4: 同じ仕事に対する費用の比。粒度が細かいほどワーカーズ型が有利になり、粒度 1 反復では 1.49 倍の差がつく。粒度 200 反復（ $\simeq 140 \mu\text{s}$ の仕事）を超えると差は 2% 以下に埋もれる。

9.3 配布コストの分離

粒度が小さい領域 ($\leq 1,000$) で $us/msg = a + b \times \text{粒度}$ に当てはめると:

	配布コスト a [$\mu s/msg$]	傾き b [ns/反復/msg]
ワーカーズ型	2.28	697.2
対称 SMP 型	4.71	699.4
差	2.43 (2.06 倍)	0.3% (一致)

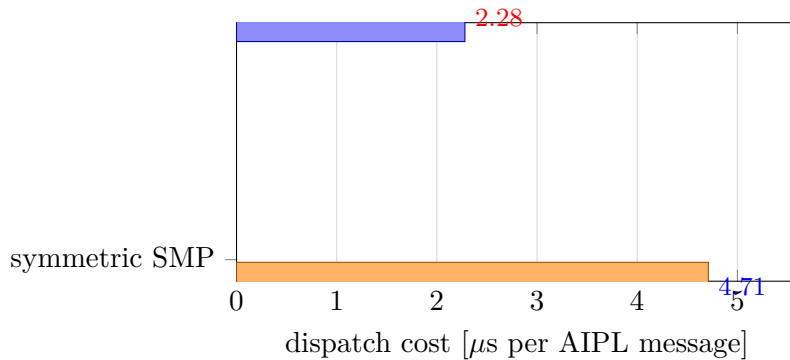


図 5: AIPL メッセージ 1 通を配るのに要する費用。本体の実行時間は含まない。

■傾き b の読み方 (訂正)。 b を「解釈器 1 反復の所要時間」と読んではいけません。測っているのはバッチ全体の壁時計時間をバッチ内メッセージ数で割った値なので、バッチが 4 コアで同時に走るぶん b には $1/4$ が掛かっている。すなわち解釈器の素の速度は $b \times 4 \simeq 2.79 \mu s/\text{反復}$ である。(この $2.79 \mu s$ は第 11 節で独立に検証される。) 初稿ではここを取り違えて「バッチが直列に走っている」と述べたが、第 10 節のバッチ長掃引がそれを否定した。

b が両方式で一致していることが重要である。本体の実行速度は同じなのだから、 a の差は配布機構そのものの費用である。ワーカーズ型は 1 バッチにつき郵便箱への書き込みと SEV 一回で済み、その費用がバッチ内の 8 通に薄まる。対称 SMP 型は 1 単位ごとにプロセスを作り、共有キューのロックを取り、文脈を切り替える。

10 結果 4: バッチ長と並列度 (実験 2)

この節の数値についての断り。本節の測定は駆動源を入れる前に採ったもので、第 4 節で述べたとおり読み込み直後の 2 ラウンド (冷間始動の過渡) しか捉えていない。定常値は約 2 倍である。また正誤を確認していない。したがって絶対値は採用しない。一方、同じ条件で二方式を交互に測った比と、コア別の実行数 `core_hits` が示す構造的な事実は、定常値で測り直した第 11 節・第 13 節と整合しており、そのまま残す。

前節の粒度掃引はバッチ長を 8 に固定していた。バッチ長そのものを振ると、AIPL のアクター・バッチが本当に 4 コアで同時に走っているかが分かる。標本は `AiplPn.abcl` (アクター n 個・各 1,000 反復・粒度は全て同じ)。

バッチ長	msgs	巡回数	1 メッセージあたり [μ s]			1 巡 [μ s]	
			W	S	S/W	W	S
2+2	4	2	1072.5	1093.0	1.019	2145	2186
4+4	8	2	686.0	690.9	1.007	2744	2764
8+8	16	4	699.1	705.7	1.009	2796	2823
16+16	32	8	740.3	746.8	1.009	2961	2987
32+23	55	14	581.7	590.3	1.015	2285	2319

「巡回数」は $\lceil \text{バッチ長}/4 \rceil$ 、すなわち 4 コアで何巡すればそのバッチを配り終えるかである。W = ワーカーズ型、S = 対称 SMP 型。

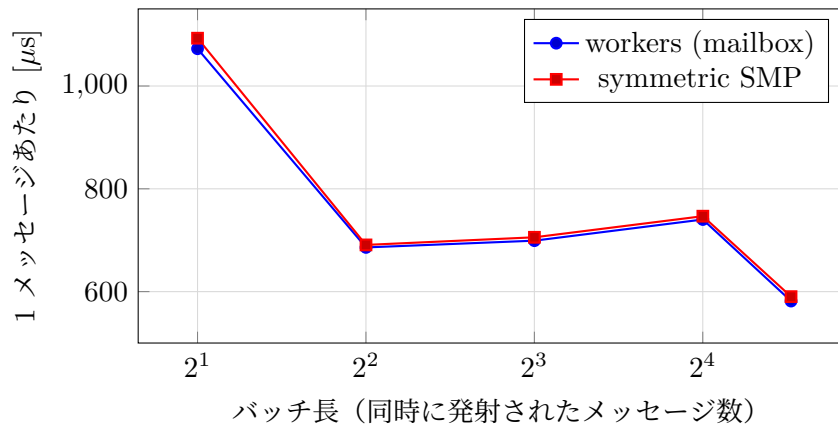


図 6: バッチ長を変えたときの 1 メッセージあたりの費用。バッチ長 2 (コア数 4 に満たない) では約 2 倍高く、4 以上ではほぼ一定になる。

■読み取れること。

1. バッチはコア数まで並列に走っている。バッチ長 2 では $1,072 \mu\text{s}/\text{msg}$ 、4 では $686 \mu\text{s}/\text{msg}$ 。半分に落ちるのは、長さ 2 のバッチが 4 コアのうち 2 コアしか使えていないからである。直列に走っているならバッチ長を変えても $\mu\text{s}/\text{msg}$ は変わらないはずで、この落ち込みが並列の証拠になる。
2. 「1 巡」の欄 (1,000 反復を全コア同時に走らせた所要時間) は $2,100 \sim 3,000 \mu\text{s}$ にほぼ収まる。
⇒ 解釈器 1 反復 $\simeq 2.8 \mu\text{s}$ 。前節の傾き $b \simeq 700 \text{ ns}$ の 4 倍で、辻褄が合う。
3. 方式差はどのバッチ長でもほぼ一定 ($S/W = 1.007 \sim 1.019$)。この粒度 ($1,000 \text{ 反復} \simeq 2.8 \text{ ms}$) では配布コストは埋もれる。方式差が効くのは第 9 節で見たとおり粒度が小さいときである。

(バッチ長 32 の標本は実測でバッチが 32 + 23 に割れており、他の行と同列には読めない。再測が必要な行として残す。)

11 結果 5-8 (改訂) : 定常値で測り直した実験 A・B・C

第 4 節で述べた 3 つの穴を直したうえで、実験 A・B・C を測り直した。以下はすべて暖機を捨てた定常値であり、全点 $\text{ng}=0$ (各ラウンドの合計が検算に通ったこと) を確認してある。方式は

ワークス型（郵便箱・静的分割）。対称 SMP 型については第 15 節を見よ。

11.1 実験 A: 交差点はどこか（ばらつき比の掃引）

総仕事量を 128,000 反復に固定し、8 アクターへの配分のばらつき比だけを振る。

比	Pi 5 (A76)		Pi 4 (A72)		予測比 (静的 makespan)
	ms/batch	比	ms/batch	比	
1	186.0	1.00	253.8	1.00	1.00
2	248.1	1.33	—	—	1.32
4	308.7	1.66	412.1	1.62	1.65
8	365.1	1.96	—	—	1.98
16	415.7	2.23	557.3	2.20	2.28
32	460.1	2.47	—	—	2.57
64	498.8	2.68	671.3	2.64	2.81
128	534.3	2.87	715.2	2.82	3.01

A76 と A72 で比の曲線がほぼ一致する（最遠点で 2.87 対 2.82）。静的分割の makespan モデルはアーキテクチャに依らない性質である。絶対値は Pi 5 のほうが 1.2~1.4 倍速い。

11.2 実験 B: 細かく割れば静的分割は救えるか

同じ重み集合（1.. M を正規化、合計 128,000 反復）を、散らした場合 B1（ビット反転順）と後ろに固めた場合 B2（単調増加）で比べる。

	$M = 4$	$M = 8$	$M = 16$
Pi 5 B1 散らす	301.5 ms	255.5 ms	235.3 ms
Pi 5 B2 固める	301.5 ms	322.6 ms	349.4 ms
Pi 4 B1 散らす	408.9 ms	339.1 ms	300.2 ms
Pi 4 B2 固める	410.6 ms	424.6 ms	434.7 ms

両板で同じ向きに分かれた。郵便箱は区間を連続に 4 等分するので、重い仕事が散らばっていれば M を増やすほど平均化されて速くなり（Pi 5: 301.5 $\rightarrow$ 235.3、Pi 4: 408.9 $\rightarrow$ 300.2）、固まっていれば同じコアに集まり続けて遅くなる（Pi 5: 301.5 $\rightarrow$ 349.4、Pi 4: 410.6 $\rightarrow$ 434.7）。

処方は「共有 ready キューへ移れ」ではない。多めに分割し、重い仕事を隣り合わせにするな — 郵便箱方式のまま実行できる。

11.3 実験 C: 4 コアで 4 倍は出ない

仕事を完全に等量（1 通 32,000 反復）にしたまま、バッチ長で使用コア数だけを変える。

使用コア	Pi 5 (A76)		Pi 4 (A72)	
	ms/batch	1 反復	ms/batch	1 反復
2	145.3	4.54 μ s	215.3	6.73 μ s
3	164.1	5.13 μ s	232.8	7.28 μ s
4	185.5	5.80 μs	255.0	7.97 μs
2→4 コアの劣化	+27.7%		+18.4%	
実効スループット	1.57× (理想 2.00)		1.69× (理想 2.00)	

分割の不均等はゼロなのに、コアを増やすと 1 反復の所要が伸びる。両板とも D-cache を切って走っているのに、すべてのメモリ参照がバスに出る。**A72 のほうが劣化が小さい** (+18.4% 対 +27.7%) のは、素の 1 反復が遅い (6.73 対 4.54 μ s) ぶんメモリ帯域への要求が緩いためと読める。N-Queens が 4 コアで 2.2× 止まりだった主因はここにある。

11.4 不揃いな仕事 (旧・実験 3)

AiplUneven (8 アクター、 $1,000 \cdot 2^k$ 反復、合計 255,000) の定常値は Pi 5 が 1,065.5 ms/batch、Pi 4 が 1,424.6 ms/batch。対称 SMP 型との比較は、対称側が正しく動かないため取れていない (第 15 節)。初稿にあった「対称 SMP 型が 1.25 倍速い」は撤回する (第 12 節)。

12 撤回: 「不揃いな仕事では対称 SMP 型が 1.25 倍速い」

初稿の第 9 節 (実験 3) で、AiplUneven について対称 SMP 型が 1.25 倍速い (403.1 ms 対 505.1 ms) と報告した。これを撤回する。理由は 2 つある。

1. 定常値ではなかった。その測定は駆動源が無い状態で採られており、実際には読み込み直後の 2 ラウンドしか走っていない。同じ標本の定常値は Pi 5 で 1,065.5 ms/batch であり、約 2 倍である。
2. 正誤を確認していなかった。検算計器を入れて測り直すと、対称 SMP 型の AIPL 経路は誤った答えを返す回がある (第 15 節)。

計算が誤っているなら、速さに意味は無い。「静的分割は最も重い区間に律速される」という理論的な指摘そのものは依然として正しく、実験 B はその影響を実測している。しかしこの実装で「対称 SMP 型のほうが速い」を実測できたとは言えない。

13 結果 9: 三板での追試 (Pi 4 / Pi 3)

Pi 5 で得た「分割の仕方が効く」という結論を、別アーキテクチャで確かめる。Pi 4 (Cortex-A72) と Pi 3 (Cortex-A53) には対称 SMP スケジューラは無いので、ここで比べるのはワーカース型 (郵便箱・静的分割) における分割の仕方である。

13.1 N-Queens $n = 13$

コア	Pi 5 (A76)	Pi 4 (A72)	Pi 3 (A53)
1	660.1 ms (1.00×)	1401.3 ms (1.00×)	1426.1 ms (1.00×)
2	355.8 ms (1.85×)	959.3 ms (1.46×)	789.0 ms (1.81×)
3	306.0 ms (2.16×)	859.0 ms (1.63×)	641.6 ms (2.22×)
4	298.8 ms (2.21×)	847.9 ms (1.65×)	517.4 ms (2.76×)

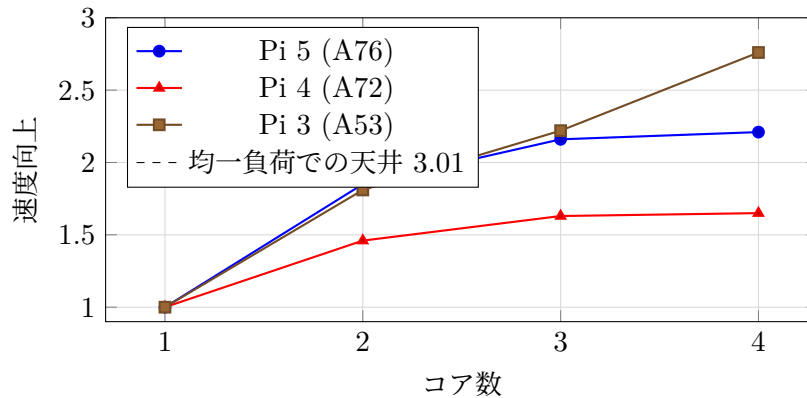


図 7: N-Queens $n = 13$ の速度向上 (各 3 回の中央値、ばらつきは全点 0.1% 未満)。**Pi 4 は 3 コアで頭打ちになるのに対し、Pi 3 は 4 コア目でも伸びる。**

Pi 3 と Pi 4 は直列がほぼ同じ (1426 対 1401 ms) なのに、4 コアでは Pi 3 が 1.6 倍速い。素直に読めば「A53 のほうが並列に強い」となるが、それは誤りである。なお **Pi 4 は 3 コアで頭打ちになる (1.63× → 1.65×)**。連続分割で重い列が同じコアに固まると、コアを足しても効かなくなる——第 11 節と同じ形である。

13.2 均一な負荷なら三板とも同じ

分割の不均衡がほとんど無い負荷 (primes, $n = 300,000$) で測り直す。

コア	Pi 4	Pi 3
1	0.99×	1.00×
2	1.61×	1.61×
3	2.31×	2.31×
4	3.01×	3.01×

曲線が完全に一致した (Pi 5 の primes も 3.02×)。したがって N-Queens の差はハードウェアの差ではなく、各カーネルが N-Queens をどう分割しているかの差である。

13.3 分割の仕方だけを変える

Pi 4 は `/bench?il=` で分割を選べる (各 2 回、再現性あり)。

分割	4 コア [ms]	速度向上
i1=0 連続 4 等分	847.6 / 847.3	1.64×
i1=1 スライド (第 1 行)	811.1 / 811.1	1.72×
i1=2 スライド (第 2 行・Pi 3 と同じ方式)	774.3 / 774.4	1.81×

スケジューラを一切変えず、分割の仕方を変えるだけで +10%。第 11 節（重い仕事を散らせば静的分割は救える）の、**実機・別アーキテクチャでの追認**である。なお Pi 3 は i1= を受け付けず既定の分割ひとつだけだが、それで 2.75× 出る。Pi 4 の i1=2 が本当に Pi 3 と同じ分割なら 1.81× 対 2.75× の差が残るので、別要因（D-cache の扱い、L2 の共有の仕方）を疑う必要がある。ここは未解決である。

13.4 Pi 3 は「アクター＝スレッド」という別の方式だった

Pi 3 に AIPL のバッチ配布を移植しようとして、**移植すべきものが無い**と分かった。Pi 3 の AIPL アクターは **Xinu スレッド＋メールボックス**である（objects[i].tid(tid_typ)、semcreate(0)、アクターごとの consumer スレッド）。バイトコード VM はそのスレッド機構の上に載っており（VM クラスは VM_CLASS_BASE=200 以降の class_id を取る）、「メッセージを束ねてコアへ配る」段階が存在しない。**Pi 3 は最初からアクター＝共有 ready キュー上のスレッド**であり、構造としては対称 SMP 型に相当する。

Pi 3 にも同じ計器（/avm-par, ok/ng, core_hits）と駆動源（/avm-run）を移植して測ったところ、次の 2 つが分かった。

■(1) 1 通も他コアへ配られない。core_hits は常に N/0/0/0 である — 4 アクターでも 16 アクターでも、すべてのメッセージが核 0 で実行される。アクターがスレッドであっても、**この Xinu はスレッドをコア間に分散しない**。多コアを使うのは smp_parallel_sum を明示的に呼ぶ経路（/bench）だけである。同じ標本で Pi 4 / Pi 5 は N/N/N/N と均等に散る。

■(2) 計算の重いアクターを 8 個載せると系が止まった。AiplP8 を載せた直後から HTTP がまったく応答しなくなった（ping は 100% 通るのでネットワークは生存）。原因は VM アクターの優先度が default: INITPRIO(20) で Web サーバ（webactor_server、同じく INITPRIO）と同値だったことである。VM アクターだけ INITPRIO-5 に下げて解消した。**アクター＝データ方式では VM が協調的に tick するので、優先度の設計を誤っても系は止まらない**。

■**定量比較は保留する**。Pi 3 の us は使えない（複数のディスパッチが同時に計時中になり、壁時計の区間が重なって 2 秒の窓に 21.4 秒を計上する）。共通の物差しは ok だが、**Pi 3 は暖機後 11 ラウンドで止まる**という未解決の問題があり（第 17 節）、スループットを Pi 4 と並べるのは時期尚早である。確実に言えるのは上の (1) と (2) の**構造的な事実**に限る。

13.5 4 コアで 4 倍は出ない、も三板に共通

均一な負荷でも 4 コアで 3.01×（Pi 5 も 3.02×）。分割の不均等がゼロでもここが天井であることは、第 11 節と整合する。

14 結果 10: Pi 4 にも同じ例外復帰の穴があった

第 5 節で Pi 5 に見つけた穴（例外エントリが ELR_EL1/SPSR_EL1 を退避しない）が、Pi 4 にもそのまま存在した。

- `irq_dispatch_c` → `proc_preempt()` → `proc_resched()` → 割り込み区間の中で `ctxsw`
- `irq_entry` は `x0-x30` と `q0-q31` まで退避するのに `ELR_EL1/SPSR_EL1` は退避しない
- その場のコメント自身が「`irq_dispatch_c` は `ctxsw` を呼びうるので割り込まれた側の FP 状態は生き延びねばならない」と書いている。同じ理屈が `ELR/SPSR` にも当てはまるのに抜けていた。

14.1 計器そのものを先に検証する

最初の測定は `mismatch=0`、つまり穴が無いように見えた。しかしそれは Pi 4 が先取り (`preemption`) を既定で切っているためで、危険が起こり得ない状態を測っていたのである。

<code>/netpreempt</code>	<code>net_preempt=off</code> (既定)
<code>/rtos-jitter preempt=0</code>	<code>pp_calls=1296 off=1296 fired=0</code> ← 一度も発火しない
<code>/rtos-jitter preempt=1</code>	<code>pp_calls=605 off=0 fired=605</code> ← 発火する

14.2 同じ起動のまま A/B を取る

`/elrfix?on=0|1` で復帰時の書き戻しを実行時に切り替えられるようにし、`/preempttest?procs=&n=` で答えの決まった計算（積・和・剰余のみ、メモリを持たない）を複数プロセスで同時に回して基準値と突き合わせた。先取りは ON。

プロセス数	修正なし <code>elr_restore_on=0</code>	修正あり <code>=1</code>
2	991055 906373 → mismatch 2/2	全て 480003 → 0
4	428433 258890 504716 480003 → 3/4	全て 480003 → 0
8	8 個すべて相異なる値 → 8/8	全て 480003 → 0

期待値は 480003。先取りが入った瞬間に、修正なしでは答えが全部狂う。修正あり側は N-Queens も 1397.2 / 960.8 / 847.8 ms・解 73712・agree = yes と焼く前に一致し、退行は無い。

14.3 Pi 3 に同じ穴は無い — ARM32 の実装は正しかった

Pi 3 の実機カーネルは `kernel7.img`、すなわち **ARMv7 (32 bit)** で、`xinu-rpi4/xinu-rpi5` とは別系統 (classic Embedded Xinu) である。AArch64 の `ELR_EL1/SPSR_EL1` に相当するのはバンクされた `LR_irq/SPSR_irq` で、その IRQ 入口はこうなっている：

```
sub lr, lr, #4
srsdb #ARM_MODE_SYS! /* SPSR_irq と LR_irq を **SYS スタック** へ積む */
cpsid if, #ARM_MODE_SYS /* = 割り込まれたスレッド自身のスタック */
push {r0-r4, r12, lr}
```

```

bl    dispatch      /* この中で clkhandler -> resched() が起こりうる */
pop   {r0-r4, r12, lr}
rfeia sp!           /* 自分のスタックから PC と CPSR を戻す      */

```

復帰状態を「割り込まれたスレッド自身のスタック」に積んでいるので、dispatch() の中で文脈が切り替わっても、各スレッドは自分の復帰状態を持って戻る。clkhandler.c は実際に割り込み中に resched() を呼ぶが、それでも安全である。

つまり AArch64 版 (rpi4/rpi5) は、ARM32 版が正しくやっていたことを再実装の際に落していた。Pi 3 は対象外であり、焼き直しも要らない。

14.4 三板の測定状況

実験	Pi 5	Pi 4	Pi 3
N-Queens コア 1-4	済	済	済
均一負荷 primes コア 1-4	済	済	済
分割の仕方 il=	口あり	済	対象外 (受け付けない)
ELR/SPSR の穴	済 (発見・修正)	済 (実証・修正)	対象外 (ARM32 で元から正しい)
dining / fill	済	済	未対応
AIPL 実験 A/B/C	済	未実装	未実装
対称 SMP 型	済	未移植	未移植
実験 1 (核 0 の参加)	済	対象外	対象外

Pi 3 / Pi 4 で残っているのは **AIPL のアクター並列**と**対称 SMP 型**の 2 つだけで、どちらも機構そのものが無いため、実装 (移植) が先である。Pi 3 は classic Embedded Xinu の系統なので、Pi 5 からの移植は Pi 4 より重い。

15 対称 SMP 型の AIPL 経路は動かなかった — その診断

検算計器を入れて対称 SMP 型で AIPL を測ると、壊れていることが直ちに分かった。

15.1 症状

標本	結果	core_hits
AiplSpread1	ok=37 ng=28	178/128/126/128 (偏り)
AiplSpread4	ok=2 ng=78	219/186/116/115
AiplEq4	ok=0、1 バッチに 4,150 ms	1/1/1/1

同じ標本を 3 回繰り返すと回ごとに挙動が変わる — 正しく速く走る回、まったく進まない回、1 バッチに 4 秒かかる回が混ざる。さらに測定中に板がネットワークごと停止した (ping も通らない)。一方 N-Queens の対称経路 (/smpsched) は正しいままであり (solutions=73712, ran_per_core=1/1/1/1, 411 ms)、壊れているのは **AIPL のバッチ配布に固有**である。

15.2 外れた仮説 1: 控えの添字の競合

`par_send` / `par_spawn` はコアごとの控えを第 3 引数で選ぶので、同時に走る実行のあいだで一意でなければならない。郵便箱方式は 1 コア 1 単位に固定するが、対称方式では単位が普通のプロセスなので、先取りで同じコアに別の単位が乗りうる — と考えて、添字を単位番号に変えた。直らなかった。

15.3 外れた仮説 2 → 部分的に当たり: 静的スタックの再利用

`sp_task` は `sp_done++` したあとも自分のスタックの上で `proc_exit()` を実行し続ける。駆動側は `sp_done` を見た瞬間に戻るなので、次の呼び出しが同じ静的スタックへ新しい文脈を書き込む。AIPL は毎秒数百回ここを通るので必ず踏み、N-Queens は HTTP 要求ごとに 1 回しか通らないので踏まない — 観測と一致する。

スタックを世代で回して（同じ領域の再利用まで 4 バッチ挟む）測り直すと:

	世代化なし	世代化あり
<code>ng</code>	28~78	0~1
<code>core_hits</code>	178/128/126/128	22/22/22/22
<code>reuse</code> (再利用時にまだ <code>PR_FREE</code> でなかった回数)	—	1~2

`reuse` が 0 でないことが、この危険が実際に起きている証拠である。世代化で誤答は激減し、配り分けの偏りも消えた。ただし主因ではなかった — 1 バッチ 4,150 ms という異常は残った。

15.4 当たり: 呼び手が自分で計算していること

時間の行き先を分解する計器を入れたところ、答えが一目で出た。

計器	値	読み
<code>wait_us</code>	1,850	生成した単位は 1.85 ms で終わっている
<code>inline_us</code>	4,150,086	呼び手の持ち分だけが 4.15 秒
<code>pp_inline</code>	399	その間に呼び手が 399 回 先取りされた
<code>reuse</code>	0	スタック再利用の危険は解消済み

$399 \times 10.4 \text{ ms}$ (1 ティック) = 4.15 秒 — `inline_us` とびたり一致する。すなわち先取り 1 回につき丸ごと 1 ティックを失っている。

並列配布そのものは健全だった (生成した単位は 1.85 ms で完了)。壊れていたのは呼び手が自分の持ち分を計算するという設計である。呼び手は普通のプロセスとして共有 ready キューに載るので、対称モードでは必ずこれを踏む。郵便箱方式では呼び手は核 0 に固定され先取りされないため、同じ設計でも問題にならない。

これは方式の差が実装の細部に現れた例である。「呼び手も 1 単位ぶん働かせる」という、静的分割では自然で効率的な設計が、動的スケジューラの上では 20 倍の劣化を招く。

15.5 現時点の状態

呼び手に持ち分を持たせない版（全単位をプロセスとして出し、呼び手は待つだけ）を実装したが、本稿の締切時点では実機での確認が済んでいない。したがって本報告では、対称 SMP 型と AIPL の比較は「実装が正しく動かないため測定できなかった」と記す。

16 考察

16.1 何が言えて、何が言えないか

本報告の測定はワーカーズ型については完結している（定常値・全点 $ng=0$ ・三板で整合）。一方対称 SMP 型と AIPL の比較は取れていない — その経路が正しく動かないためである（第 15 節）。したがって「二方式のどちらが速いか」を AIPL について定量的に述べることは、本報告ではできない。以下は言えることに限る。

16.2 仮説を支持する材料（間接的だが強い）

仮説は「AIPL のような短いメッセージには、郵便箱方式（アクター＝データ）が向く」である。直接の対決は測れなかったが、次の 3 つはいずれもこの向きを支持する。

1. 対称 SMP 型はそもそも AIPL を正しく走らせられなかった。誤答・停止・板の死が混ざる。原因は呼び手が自分の持ち分を計算すると先取りで 1 ティックずつ失うことにあり ($399 \times 10.4 \text{ ms} = 4.15 \text{ s}$)、これは短い仕事を汎用スケジューラに載せることの費用そのものである。同じコードが N-Queens（要求ごとに 1 回、重い区間）では正しく動く。壊れ方が「短く・頻繁」という条件に選択的であることが、仮説の核心と一致する。
2. アクター＝スレッドの板 (Pi 3) では、1 通も他コアへ配られない。core_hits は常に N/0/0/0（第 13 節）。アクターをスレッドにしても、それだけでは多コアは使われない。
3. Pi 3 では計算の重いアクターを 8 個載せると系が止まった。VM アクターの優先度が Web サーバと同値だったため、優先度を下げて解消した。アクター＝データ方式では VM が協調的に tick するので、優先度の設計を誤っても系は止まらない。速度以外の面でも強い。

16.3 ワーカーズ型について実測で言えること

- 静的分割の makespan モデルが成り立つ。ばらつき比を振ると実測は予測に 5% 前後で追随し、A76 と A72 で比の曲線がほぼ一致する（第 11 節）。方式の振る舞いはアーキテクチャに依らない。
- 静的分割の弱点は「分割の粗さ」であって「静的であること」ではない。重い仕事が散らばっていれば、分割数を増やすだけで理想に近づく (Pi 5: $301.5 \rightarrow 235.3 \text{ ms}$ 、Pi 4: $408.9 \rightarrow 300.2 \text{ ms}$)。固まっていれば増やすほど悪化する。処方は「多めに分割し、重い仕事を隣り合わせにするな」であり、郵便箱方式を保ったまま実行できる。
- 4 コアで 4 倍は出ない。仕事を完全に等量にしても、2→4 コアで 1 反復が Pi 5 で +27.7%、

Pi 4 で +18.4% 伸びる。D-cache を切って走っているためで、**N-Queens** が 2.2× 止まりだった主因はここにある（分割の不均等ではない）。

16.4 アーキテクチャ上の含意

アクターモデルの言語処理系では、メッセージは**データ**として配るのが安い。プロセスとして配ると、汎用スケジューラが長寿命スレッドを前提に払っている費用（生成・キュー・文脈切替・先取り）を、寿命が 100 μ s に満たない仕事にも払うことになる。本報告で対称 SMP 型が壊れた形 — **呼び手**が先取りされて **1 ティックずつ失う** — は、その費用が最も露骨に出た例である。

Xinu Pi 上の AIPL 実行基盤としては、**バッチ化した郵便箱方式を既定とし、分割数を仕事の不揃いさに応じて増やすのが妥当**と考える。動的分配の併用は、上記の実装上の問題を解いたうえで改めて評価すべきである。

17 未解決

1. 対称 SMP 型で AIPL を正しく走らせる。呼び手に持ち分を持たせない版（全単位をプロセスとして出し、呼び手は待つだけ）を実装したが、本稿の締切時点で実機確認が済んでいない。これが通れば、撤回した「不揃いな仕事での二方式の比較」を初めて正しく測り直せる。
2. 冷間始動が定常のちょうど半分になる理由（第 4 節）。描画リストの蓄積は否定済み。
3. Pi 3 が暖機後 11 ラウンドで止まる。駆動源の詰まりは直したが（窓 500/1000 ms で 14.0/13.0 round/s と一致）、窓を倍にしても ok が 11 のまま増えない。何かが枯渇している。このため Pi 3 のスループットを Pi 4 と並べるのは時期尚早である。
4. Pi 3 の us は使えない。アクター＝スレッドなので複数のディスパッチが同時に計時中になり、壁時計の区間が重なって積算される（2 秒の窓に 21.4 秒を計上）。三板共通の物差しは ok（検算に通ったラウンド数）に限る。
5. 郵便箱経路のワーカ取りこぼし。SMP_WAIT_LIMIT 超過で核 0 が肩代わりすると、同じ区間が二重に実行されうる。発火するのはワーカが詰まったときだけで、本測定では観測されていない。
6. 二方式の同居。対称モードに切り替えたあと郵便箱経路を叩くと結果が壊れる。現状は排他的に使う前提である。
7. Pi 3 の例外エントリは ARM32 で元から正しいことをソースで確認済み。実機での /preempttest 相当は未実施。

18 実行したプログラム

18.1 AIPL 標本（粒度 200 の版）

粒度だけが異なる 9 本を用意した（1, 10, 50, 200, 1,000, 5,000, 20,000, 100,000, 200,000）。以下は粒度 200 の版。bench/aipl_ab/AiplBenchG200.abcl。

```
// AiplBenchG200.abcl — AIPL のアクター並列を測る標本（粒度 200 = 1メッセージあたりのループ回数）
//
```

```
// 8 個の Worker が **同じ量** の計算をする。宛先が互いに異なり par_safe
// (WAIT も描画命令も含まない) ので、実行時系はこの 8 通を 1 バッチに
// まとめて各コアへ配る。Main.tick() は毎フレーム呼ばれるので、走らせている
// あいだ同じ形の並列バッチが流れ続ける＝スループットを測れる。
//
// ★ アクターは **最初の 1 回だけ**作り、以後は同じ相手へ送り直す。
// 毎フレーム new すると VM のオブジェクト表がすぐ埋まり、数ラウンドで
// 止まってしまう (実測でそうなった)。
//
// Collector は合計が 1600 (= 8 * 200) と一致するかを毎ラウンド確かめ、
// 一致すれば緑、違えば赤の線を引く。壊れた状態の速い数字を良い結果と
// 取り違えないための自己検査。
//
// 測り方: /avm-par?on=1&reset=1 → 一定時間走らせる → /avm-par の us と msgs。
// us/msgs が「1 メッセージあたりの費用」で、方式の比較はこの値で行う。
class Main {
  var ready = 0;
  var c = 0;
  var w0 = 0; var w1 = 0; var w2 = 0; var w3 = 0;
  var w4 = 0; var w5 = 0; var w6 = 0; var w7 = 0;
  method tick() {
    if (ready == 0) {
      ready = 1;
      c = new Collector();
      w0 = new Worker(); w1 = new Worker(); w2 = new Worker(); w3 = new Worker();
      w4 = new Worker(); w5 = new Worker(); w6 = new Worker(); w7 = new Worker();
    }
    send w0.run(c, 200); send w1.run(c, 200);
    send w2.run(c, 200); send w3.run(c, 200);
    send w4.run(c, 200); send w5.run(c, 200);
    send w6.run(c, 200); send w7.run(c, 200);
  }
}
class Worker {
  var acc = 0; var i = 0;
  method run(coll, n) {
    acc = 0;
    i = 0;
    while (i < n) do { acc = acc + 1; i = i + 1; }
    send coll.report(acc);
  }
}
class Collector {
  var total = 0; var got = 0;
  method report(v) {
    total = total + v;
    got = got + 1;
    if (got == 8) {
      got = 0;
      // 合計が合ったときだけ次ラウンドを起こす (これが無いと 1 ラウンドで止まる)。
      // 誤っていれば止まるので、/avm-par の msgs が伸びないことで外から誤りが分かる。
      if (total == 1600) { total = 0; line(80, 300, 680, 300, 2); wait(1); }
      else { total = 0; line(80, 300, 680, 300, 4); }
    }
  }
}
}
```

18.2 ばらつき比を振る標本 (実験 A・比 1:16 の版)

```
// AiplSpread16.abcl — ばらつき比 1:16 (合計固定)
//
// 実験A「交差点」用。総仕事量を固定したままばらつきだけを振り、
// ワーカーズ型 (静的分割) と対称SMP型 (動的分配) が入れ替わる点を探す。
// 予測 makespan: 静的 73100 反復 / 動的 52700 反復 (比 1.39)。
//
// ワーカ 8 個・合計 128000 反復。毎ラウンド合計を検算する。
class Main {
  var ready = 0;
  var c = 0;
  var w0 = 0;
  var w1 = 0;
  var w2 = 0;
  var w3 = 0;
  var w4 = 0;
  var w5 = 0;
```

```

var w6 = 0;
var w7 = 0;
method tick() {
  if (ready == 0) {
    ready = 1;
    c = new Collector();
    w0 = new Worker();
    w1 = new Worker();
    w2 = new Worker();
    w3 = new Worker();
    w4 = new Worker();
    w5 = new Worker();
    w6 = new Worker();
    w7 = new Worker();
  }
  send w0.run(c, 2700);
  send w1.run(c, 4100);
  send w2.run(c, 6000);
  send w3.run(c, 9000);
  send w4.run(c, 13300);
  send w5.run(c, 19800);
  send w6.run(c, 29400);
  send w7.run(c, 43700);
}
}
class Worker {
  var acc = 0; var i = 0;
  method run(coll, n) {
    acc = 0;
    i = 0;
    while (i < n) do { acc = acc + 1; i = i + 1; }
    send coll.report(acc);
  }
}
class Collector {
  var total = 0; var got = 0;
  method report(v) {
    total = total + v;
    got = got + 1;
    if (got == 8) {
      got = 0;
      // 合計が合ったときだけ次ラウンドを起こす。誤っていれば止まるので
      // /aum-par の msgs が伸びないことで **外から** 誤りが分かる。
      if (total == 128000) { total = 0; line(80, 300, 680, 300, 2); wait(1); }
      else { total = 0; line(80, 300, 680, 300, 4); }
    }
  }
}
}

```

18.3 分割の粗さを見る標本（実験 B・重い仕事が入る $M = 8$ の版）

```

// AiplB2M8.abcl — 同じ重み集合を 後ろに固めた・8 通に分割
//
// 実験B「分割を細かくすれば静的分割は救えるか」用。同じ重み集合 (1..M を正規化)を
// 後ろに固めた (単調増加)。
// 郵便箱は区間を **連続** に 4 等分するので、重い仕事が入ってれば M を増やしても
// 救えないはず。散らばってれば平均化されて理想に近づくはず。
// 予測 makespan: 静的 53300 / 動的 42600 反復 (比 1.25、理想 32000)。
// ワーク 8 個・合計 128000 反復。毎ラウンド合計を検算する。
class Main {
  var ready = 0;
  var c = 0;
  var w0 = 0;
  var w1 = 0;
  var w2 = 0;
  var w3 = 0;
  var w4 = 0;
  var w5 = 0;
  var w6 = 0;
  var w7 = 0;
  method tick() {
    if (ready == 0) {
      ready = 1;
      c = new Collector();
      w0 = new Worker();

```

```

        w1 = new Worker();
        w2 = new Worker();
        w3 = new Worker();
        w4 = new Worker();
        w5 = new Worker();
        w6 = new Worker();
        w7 = new Worker();
    }
    send w0.run(c, 3600);
    send w1.run(c, 7100);
    send w2.run(c, 10700);
    send w3.run(c, 14200);
    send w4.run(c, 17800);
    send w5.run(c, 21300);
    send w6.run(c, 24900);
    send w7.run(c, 28400);
}
}
class Worker {
    var acc = 0; var i = 0;
    method run(coll, n) {
        acc = 0;
        i = 0;
        while (i < n) do { acc = acc + 1; i = i + 1; }
        send coll.report(acc);
    }
}
class Collector {
    var total = 0; var got = 0;
    method report(v) {
        total = total + v;
        got = got + 1;
        if (got == 8) {
            got = 0;
            // 合計が合ったときだけ次ラウンドを起こす。誤っていれば止まるので
            // /aum-par の msgs が伸びないことで **外から** 誤りが分かる。
            if (total == 128000) { total = 0; line(80, 300, 680, 300, 2); wait(1); }
            else { total = 0; line(80, 300, 680, 300, 4); }
        }
    }
}
}

```

18.4 使用コア数を変える標本（実験 C・4 コアの版）

```

// AiplEq4.abcl — 等量 32000 反復 × 4 通 (=4 コアだけが働く)
//
// 実験 C「4 コアで本当に 4 倍出るのか」用。バッチ長がコア数に満たないと
// 残りのコアは 1 通も実行しない (core_hits で確認済み) ことを利用して、
// **仕事を完全に揃えたまま** 使われるコア数だけを 1→4 と変える。
// ここで 4 倍に届かないぶんは、分割の不均等ではなく D-cache OFF の
// メモリ帯域（およびロック・配布の固定費）で説明されるはず。

// ワーカ 4 個・合計 128000 反復。毎ラウンド合計を検算する。
class Main {
    var ready = 0;
    var c = 0;
    var w0 = 0;
    var w1 = 0;
    var w2 = 0;
    var w3 = 0;
    method tick() {
        if (ready == 0) {
            ready = 1;
            c = new Collector();
            w0 = new Worker();
            w1 = new Worker();
            w2 = new Worker();
            w3 = new Worker();
        }
        send w0.run(c, 32000);
        send w1.run(c, 32000);
        send w2.run(c, 32000);
        send w3.run(c, 32000);
    }
}

```

```

class Worker {
  var acc = 0; var i = 0;
  method run(coll, n) {
    acc = 0;
    i = 0;
    while (i < n) do { acc = acc + 1; i = i + 1; }
    send coll.report(acc);
  }
}
class Collector {
  var total = 0; var got = 0;
  method report(v) {
    total = total + v;
    got = got + 1;
    if (got == 4) {
      got = 0;
      // 合計が合ったときだけ次ラウンドを起こす。誤っていれば止まるので
      // /avm-par の msgs が伸びないことで **外から** 誤りが分かる。
      if (total == 128000) { total = 0; line(80, 300, 680, 300, 2); wait(1); }
      else { total = 0; line(80, 300, 680, 300, 4); }
    }
  }
}

```

18.5 核 0 を対称スケジューラに参加させる修正（実験 1）

```

/* system/proc.c --- タイマ IRQ から EOI 後に必ず呼ばれる地点。
 * 既存の「currpid が idle なら何もしない」判定こそが、核 0 が
 * 対称スケジューラに一度も参加しない原因だった。 */
void proc_preempt(void)
{
#ifdef SMP_SYMMETRIC
  /* この核が暇 (idle プロセスに居る) なら、共有 ready キューから 1 本引き受ける。 */
  {
    extern volatile int smpsched_on;
    extern int smpsched_poll_once(void);
    if (smpsched_on && IS_IDLE(currpid)) { smpsched_poll_once(); return; }
  }
#endif
  if (!g_preempt_on || !g_resched_pending) return;
  if (g_actor_pump) return;
  g_resched_pending = 0;
  if (!IS_IDLE(currpid)) proc_resched();
}

```

18.6 不揃いな仕事の標本（実験 3）

```

// AiplUneven.abcl - 仕事が **不揃い** な標本（等比: 1000 から 128000 まで倍々）
//
// 実験3「主張の境界」用。ワーカーズ型は 8 通を 4 コアへ **静的に** 2 通ずつ割る
// ので、重い 2 通を引いたコアの終了時刻が全体を決める (64000+128000 = 192000 反復)。
// 対称SMP型は各コアが空き次第 自分で取るので、走行中に均される。
// =ここは対称SMP型が有利になりうる領域であり、正直に測る。
// 合計は 255000 反復。毎ラウンド検算する。
class Main {
  var ready = 0;
  var c = 0;
  var w0 = 0;
  var w1 = 0;
  var w2 = 0;
  var w3 = 0;
  var w4 = 0;
  var w5 = 0;
  var w6 = 0;
  var w7 = 0;
  method tick() {
    if (ready == 0) {
      ready = 1;
      c = new Collector();
      w0 = new Worker();
      w1 = new Worker();

```



```

        w2 = new Worker();
        w3 = new Worker();
        w4 = new Worker();
        w5 = new Worker();
        w6 = new Worker();
        w7 = new Worker();
    }
    send w0.run(c, 1000);
    send w1.run(c, 2000);
    send w2.run(c, 4000);
    send w3.run(c, 8000);
    send w4.run(c, 16000);
    send w5.run(c, 32000);
    send w6.run(c, 64000);
    send w7.run(c, 128000);
}
}
class Worker {
    var acc = 0; var i = 0;
    method run(coll, n) {
        acc = 0;
        i = 0;
        while (i < n) do { acc = acc + 1; i = i + 1; }
        send coll.report(acc);
    }
}
class Collector {
    var total = 0; var got = 0;
    method report(v) {
        total = total + v;
        got = got + 1;
        if (got == 8) {
            got = 0;
            // 合計が合ったときだけ次ラウンドを起こす。誤っていれば止まるので
            // /aum-par の msgs が伸びないことで **外から** 誤りが分かる。
            if (total == 255000) { total = 0; line(80, 300, 680, 300, 2); wait(1); }
            else { total = 0; line(80, 300, 680, 300, 4); }
        }
    }
}
}

```

18.7 バッチ長を振る標本（実験 2・バッチ長 4 の版）

```

// AiplP4.abcl — アクター 4 個・粒度 1000。バッチ長 4 で 1 通あたりの費用を測る。
// 並列に走っているなら、コア数より少ないバッチ長では 1 通あたりが増えるはず
// (2 個なら 2 コアしか使えないので 1 通あたりは 4 個・8 個のときの倍になる)。
// 直列に走っているなら、バッチ長によらず 1 通あたりは一定になる。
// =この比較で「本当に並列に走っているか」が決まる。合計 4000。
class Main {
    var ready = 0;
    var c = 0;
    var w0 = 0;
    var w1 = 0;
    var w2 = 0;
    var w3 = 0;
    method tick() {
        if (ready == 0) {
            ready = 1;
            c = new Collector();
            w0 = new Worker();
            w1 = new Worker();
            w2 = new Worker();
            w3 = new Worker();
        }
        send w0.run(c, 1000);
        send w1.run(c, 1000);
        send w2.run(c, 1000);
        send w3.run(c, 1000);
    }
}
class Worker {
    var acc = 0; var i = 0;
    method run(coll, n) {
        acc = 0;
        i = 0;
        while (i < n) do { acc = acc + 1; i = i + 1; }
    }
}

```

```

    send coll.report(acc);
}
}
class Collector {
    var total = 0; var got = 0;
    method report(v) {
        total = total + v;
        got = got + 1;
        if (got == 4) {
            got = 0;
            // 合計が合ったときだけ次ラウンドを起こす。誤っていれば止まるので
            // /aum-par の msgs が伸びないことで **外から** 誤りが分かる。
            if (total == 4000) { total = 0; line(80, 300, 680, 300, 2); wait(1); }
            else { total = 0; line(80, 300, 680, 300, 4); }
        }
    }
}
}
}

```

18.8 N-Queens: 対称 SMP 型の駆動部

system/smpsched.c より、比較実験の中核。単位ごとにプロセスを作って共有 ready キューへ入れ、呼び手も 1 単位を直接担当する。

```

long smpsched_parallel(smp_range_fn fn, long n, int nunits)
{
    /* 再入よけ。この関数は wm ループ (核0) から呼ばれ、待ちの proc_resched() で
     * ネットワーク・プロセスが走る。そこから何かがまた並列配布を要求しても
     * 大域 (sp_res/sp_done...) を壊さないよう、入れ子は直列で処理する。 */
    static volatile int sp_busy;
    if (n <= 0) return 0;
    if (sp_busy) return fn(0, n, smp_core_id());
    if (nunits < 1) nunits = 1;
    if (nunits > SP_MAXUNIT) nunits = SP_MAXUNIT;
    if (nunits > n) nunits = (int)n;
    if (!smpsched_on || nunits == 1) return fn(0, n, smp_core_id()); /* 対称でなければ直列 */

    sp_busy = 1;
    sp_fn = fn;
    sp_done = 0;
    for (int i = 0; i < SMP_NCORES; i++) sp_ran[i] = 0;

    int pids[SP_MAXUNIT], nproc = 0;
    for (int t = 0; t < nunits - 1; t++) { /* 最後の 1 単位は呼び手が直接走らせる */
        int pid = proc_create_static_susp(sp_task, sp_stk[t], SP_STK, "sptask");
        if (pid < 0) break;
        pids[nproc] = pid;
        sp_lo[pid] = (int)((long)t * n) / nunits;
        sp_hi[pid] = (int)((long)(t + 1) * n) / nunits;
        sp_res[pid] = 0;
        nproc++;
    }
    for (int t = 0; t < nproc; t++) proc_ready(pids[t]);

    unsigned long t0 = now_us();
    /* 呼び手の持ち分 = [nproc*n/nunits, n)。作れなかったぶんもここに入るの
     区間は必ずちょうど一度だけ覆われる。 */
    long total = fn((long)((long)nproc * n) / nunits, n, smp_core_id());
    sp_ran[smp_core_id()]++; /* 呼び手の持ち分も数える */
    while (sp_done < nproc) proc_resched();
    sp_us = (long)(now_us() - t0);
    sp_calls++; sp_units_tot += nunits;

    for (int t = 0; t < nproc; t++) total += sp_res[pids[t]];
    sp_busy = 0;
    return total;
}

```

18.9 ワーカーズ型の配布 (郵便箱・静的分割)

system/smp.c の smp_parallel_sum。

```

long smp_parallel_sum(smp_range_fn fn, long n, int ncores)
{
    if (ncores < 1) ncores = 1;
    if (ncores > SMP_NCORES) ncores = SMP_NCORES;
    if (n < 0) n = 0;

    long chunk = n / ncores;
    long total = 0;

    /* Post chunks 1..ncores-1 to the worker cores (skip offline ones — those
     * chunks are computed by core 0 below). */
    for (int c = 1; c < ncores; c++) {
        /* ★ 剰余を「最後のコア」に丸投げしない。chunk = n/ncores の整数除算で
         余りを c==ncores-1 に全部載せると、n がコア数で割り切れないとき
         その 1 コアだけが余りを抱え、makespan が跳ねる (実測: 4 コアに 3 列を
         渡すと 1 コアが 3 列を直列処理し、4 列を渡したときより遅かった)。
         lo = c*n/nc, hi = (c+1)*n/nc なら余りは各コアへ 1 つずつ散る。 */
        long lo = ((long)c * n) / ncores;
        long hi = ((long)(c + 1) * n) / ncores;
        if (!smp_online[c]) { total += fn(lo, hi, 0); continue; }
        smp_job_fn[c] = fn;
        smp_job_lo[c] = lo;
        smp_job_hi[c] = hi;
        MB_CLEAN(&smp_job_fn[c]); MB_CLEAN(&smp_job_lo[c]); MB_CLEAN(&smp_job_hi[c]);
        dsb();
        smp_job_seq[c]++; /* arm the job, then wake the worker */
        MB_CLEAN(&smp_job_seq[c]);
        dsb_sev();
    }

    /* Core 0 runs chunk 0 inline while the workers run theirs. */
    total += fn(0, (ncores == 1) ? n : chunk, 0);

    /* Collect the workers, taking over any that did not finish in time. */
    for (int c = 1; c < ncores; c++) {
        if (!smp_online[c]) continue; /* already done inline above */
        unsigned long spins = 0;
        MB_INVALID(&smp_job_done[c]);
        while (smp_job_done[c] != smp_job_seq[c]) {
            if (++spins >= SMP_WAIT_LIMIT) { /* worker stuck — do it here */
                /* ★ 剰余を「最後のコア」に丸投げしない。chunk = n/ncores の整数除算で
                 余りを c==ncores-1 に全部載せると、n がコア数で割り切れないとき
                 その 1 コアだけが余りを抱え、makespan が跳ねる (実測: 4 コアに 3 列を
                 渡すと 1 コアが 3 列を直列処理し、4 列を渡したときより遅かった)。
                 lo = c*n/nc, hi = (c+1)*n/nc なら余りは各コアへ 1 つずつ散る。 */
                long lo = ((long)c * n) / ncores;
                long hi = ((long)(c + 1) * n) / ncores;
                smp_job_res[c] = fn(lo, hi, 0);
                break;
            }
            __asm__ volatile("nop");
            MB_INVALID(&smp_job_done[c]);
        }
        MB_INVALID(&smp_job_res[c]);
        total += smp_job_res[c];
    }
    return total;
}

/* =====
 * Boot-time self-test / benchmark: count primes in [0,n) on 1 core,
 * then on all online cores, and print wall-clock times + speedup to
 * the serial console. Lets us compare 1-core vs 4-core scaling even
 * with no network up. Also exposed over HTTP via /smp-bench.
 * ===== */

extern void uart_puts(const char *s);
extern void uart_putc(char c);

/* Print an unsigned decimal to the UART. */
static void uart_dec(unsigned long v)
{
    char buf[24];
    int i = 0;
    if (v == 0) { uart_putc('0'); return; }
    while (v && i < (int)sizeof buf) { buf[i++] = (char)('0' + (v % 10)); v /= 10; }
    while (i--) uart_putc(buf[i]);
}

```

```

/* Free-running counter in milliseconds (CNTPCT_ELO / CNTFRQ_ELO * 1000). */
static unsigned long smp_now_ms(void)
{
    unsigned long ct, hz;
    __asm__ volatile ("mrs %0, cntpct_el0" : "=r"(ct));
    __asm__ volatile ("mrs %0, cntfrq_el0" : "=r"(hz));
    if (!hz) return 0;
    return (ct * 1000UL) / hz;
}

/* Prime-count kernel (trial division) — pure integer compute, matches
 * smp_range_fn so smp_parallel_sum can split it across cores. */

```

18.10 AIPL アクター・バッチの配り分け

device/video/avm.c。実行時にどちらの方式で配るかを決める。

```

par_batch = batch; par_active = 1;
    avm_par_nbatch++; avm_par_lastbn = bn; /* diag */
    /* ★ AIPL のアクター・バッチをどちらの方式で配るかは実行時に決まる。
     * 対称モード (/smpmode?on=1) では共有 ready キューへ、
     * それ以外は従来どおり郵便箱（静的分割）へ。
     * 対称モードで郵便箱に落とすと壊れる（実測：82 秒・解が不一致）ので、
     * ここを分けないと「AIPL を対称SMP型で並列実行」ができない。 */
    {
        unsigned long d0 = avm_now_us();
#ifdef SMP_SYMMETRIC
        extern volatile int smpsched_on;
        extern long smpsched_parallel(long (*)(long, long, int), long, int);
        if (smpsched_on) smpsched_parallel(par_dispatch_range, bn, nc);
        else
            smp_parallel_sum(par_dispatch_range, bn, nc);
#else
        smp_parallel_sum(par_dispatch_range, bn, nc);
#endif
        avm_par_us += (long)(avm_now_us() - d0);
        avm_par_msgs += bn;
    }
    par_active = 0;

```

18.11 不具合の修正（例外の ELR/SPSR 退避）

system/exception_vectors.S。

```

.global irq_entry
irq_entry:
    sub    sp, sp, #(16 * 17)
    stp    x0, x1, [sp, #16 * 0]
    stp    x2, x3, [sp, #16 * 1]
    stp    x4, x5, [sp, #16 * 2]
    stp    x6, x7, [sp, #16 * 3]
    stp    x8, x9, [sp, #16 * 4]
    stp    x10, x11, [sp, #16 * 5]
    stp    x12, x13, [sp, #16 * 6]
    stp    x14, x15, [sp, #16 * 7]
    stp    x16, x17, [sp, #16 * 8]
    stp    x18, x19, [sp, #16 * 9]
    stp    x20, x21, [sp, #16 * 10]
    stp    x22, x23, [sp, #16 * 11]
    stp    x24, x25, [sp, #16 * 12]
    stp    x26, x27, [sp, #16 * 13]
    stp    x28, x29, [sp, #16 * 14]
    str    x30, [sp, #16 * 15]
    // ★ ELR_EL1 / SPSR_EL1 も退避する。この核は割り込みハンドラの中で
    // 文脈切替(proc_preempt -> proc_resched -> ctxsw)を行うため、
    // 「戻り先が未確定の例外」が同時に複数存在しうる。保存しないと
    // 後から入った例外が上書きし、別の戻り先へ eret して計算が壊れる
    // (実測：計算プロセス2本以上のときだけ結果が毎回変わった)。
    mrs    x0, elr_el1
    mrs    x1, spsr_el1
    stp    x0, x1, [sp, #16 * 16]

```

```

bl      irq_dispatch_c

ldp     x0, x1, [sp, #16 * 16]    // 先に ELR/SPSR を戻す (x0/x1 はこの後で上書きされる)
msr     elr_el1, x0
msr     spsr_el1, x1
ldp     x0, x1, [sp, #16 * 0]
ldp     x2, x3, [sp, #16 * 1]
ldp     x4, x5, [sp, #16 * 2]
ldp     x6, x7, [sp, #16 * 3]
ldp     x8, x9, [sp, #16 * 4]
ldp     x10, x11, [sp, #16 * 5]
ldp     x12, x13, [sp, #16 * 6]
ldp     x14, x15, [sp, #16 * 7]
ldp     x16, x17, [sp, #16 * 8]
ldp     x18, x19, [sp, #16 * 9]
ldp     x20, x21, [sp, #16 * 10]
ldp     x22, x23, [sp, #16 * 11]
ldp     x24, x25, [sp, #16 * 12]
ldp     x26, x27, [sp, #16 * 13]
ldp     x28, x29, [sp, #16 * 14]
ldr     x30, [sp, #16 * 15]
add     sp, sp, #(16 * 17)
eret

```

18.12 測定スクリプト

```

#!/usr/bin/env bash
# measure.sh --- AIPL のアクター並列を粒度別に測る (1 ロード = 数ラウンド)。
#
# VM の tick はロード時に一度だけ投入され、Collector の wait(1) が次のラウンドを
# 起こす。そこで「計器を 0 にする → 標本を読み込む → 一定時間待つ → 読む」を
# 1 標本とし、us と msgs から **1 メッセージあたりの費用** を出す。
#
# 使い方: ./measure.sh <host> <ラベル> [待ち秒] → TSV を stdout へ
# 例:      ./measure.sh 192.168.3.101 workers 4 > workers.tsv
set -u
HOST="${1:-192.168.3.101}"
LABEL="${2:-run}"
WAIT="${3:-4}"
GRAN="1 10 50 200 1000 5000 20000 100000 200000"
REPS="${4:-2}"

printf "# host=%s label=%s wait=%ss reps=%s\n" "$HOST" "$LABEL" "$WAIT" "$REPS"
printf "# %s\n" "$(curl -s -m 10 "http://$HOST/version" | head -1)"
printf "# %s\n" "$(curl -s -m 10 "http://$HOST/smpmode" | head -1)"
printf "granularity\ttrep\tus\tmsgs\tbatches\tus_per_msg\tpath\n"

for g in $GRAN; do
  avm="/tmp/AiplBenchG$g.avm"
  [ -f "$avm" ] || continue
  len=$(stat -f%z "$avm" 2>/dev/null || stat -c%s "$avm")
  for r in $(seq 1 "$REPS"); do
    curl -s -m 10 "http://$HOST/avm-par?on=1&reset=1" >/dev/null
    curl -s -m 20 -X POST --data-binary @"$avm" "http://$HOST/actor/loadvm?off=0" >/dev/null
    curl -s -m 20 "http://$HOST/actor/loadvm?go=1&len=$len" >/dev/null
    sleep "$WAIT"
    out="$(curl -s -m 10 "http://$HOST/avm-par")"
    us=$(echo "$out" | sed -n 's/.* us=([0-9]*).*$/\1/p')
    ms=$(echo "$out" | sed -n 's/.* msgs=([0-9]*).*$/\1/p')
    nb=$(echo "$out" | sed -n 's/.* nbatch=([0-9]*).*$/\1/p')
    pa=$(echo "$out" | sed -n 's/.* path=([a-z]*).*$/\1/p')
    if [ -n "${ms:-}" ] && [ "${ms:-0}" -gt 0 ]; then
      upm=$(python3 -c "print(f'${us}/${ms:.3f}')" )
    else
      upm="NA"
    fi
    printf "%s\t%s\t%s\t%s\t%s\t%s\t%s\n" "$g" "$r" "${us:-NA}" "${ms:-NA}" "${nb:-NA}" "$upm" "${pa:-NA}"
  done
done
done

```